

Learning with AI

Topic 2: Mobile App Development with React/PWA

Mausam Shrestha

CSC 494 — IoT | Spring 2026

Northern Kentucky University

Software Domain — How I built a real-time IoT dashboard accessible from any device

What Problem Did I Need to Solve?

My Smart Home Health Monitor generates sensor data 24/7.

People need to **see that data anywhere, from any device** — without installing an app.

Requirements:

- Real-time updates (data changes every 60 seconds)
- Historical charts (24 hours, 7 days, 30 days)
- Works on phone AND desktop
- No App Store required
- Automatic purifier control status
- Sleep quality logging interface

Solution: React + Progressive Web App + Firebase

What I Didn't Know Before

Going into Sprint 2, my web experience was basic HTML/CSS/JavaScript:

- Never built a React application
- Did not understand component-based UI architecture
- Never worked with real-time databases
- Did not know what a PWA or service worker was
- Never built a data visualization dashboard
- Did not understand state management in reactive apps

This topic pushed me significantly beyond my starting point.

Why React?

AI Helped Me Understand the Right Tool for This Job

My first instinct: Just use plain JavaScript and `setInterval` to refresh the page.

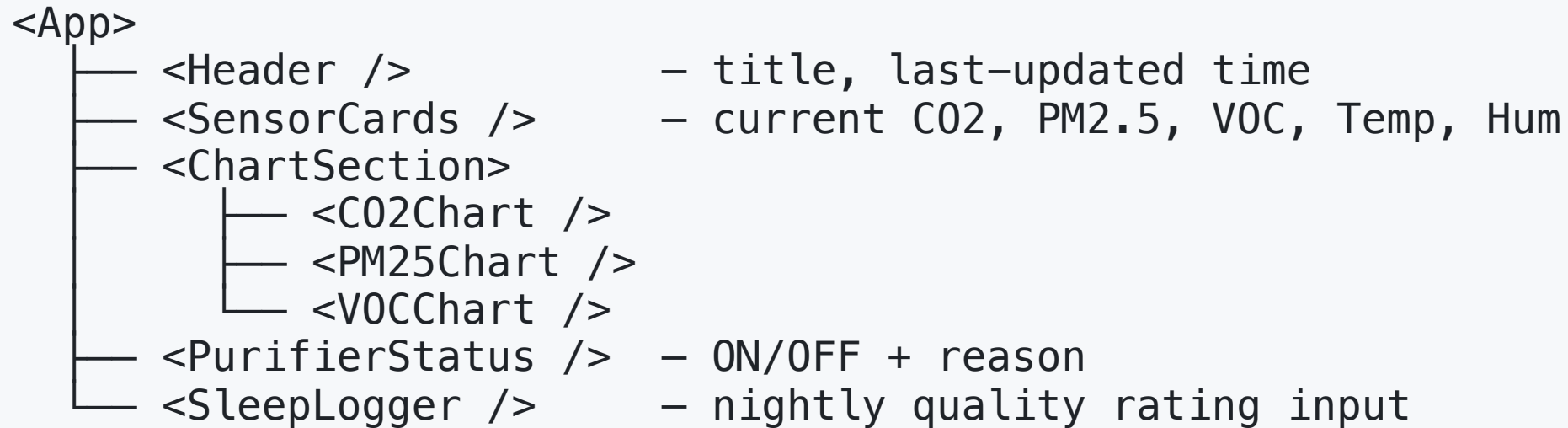
AI's explanation of why that's wrong for IoT:

- Refreshing the whole page causes flicker and loses user context
- `setInterval` polling wastes bandwidth and adds unnecessary latency
- React's component model means **only the chart that has new data re-renders**

Key insight AI gave me: In a reactive application, the UI is a *function of state*. When state changes (new sensor reading arrives), React automatically figures out the minimum DOM update needed. You don't tell it *how* to update — you tell it *what the data is*.

React Architecture I Learned

Components Are Building Blocks



Each component:

- Receives data as props
- Manages its own local state if needed

Firestore Real-Time Integration

The Most Important Thing I Learned

Static databases: You request data → you get data → done.

Firestore with `onSnapshot`: You *subscribe* → Firestore pushes every change to you instantly.

```
// This runs ONCE – but the callback fires every time data changes
const unsubscribe = db.collection("sensor_readings")
  .orderBy("timestamp", "desc")
  .limit(100)
  .onSnapshot((snapshot) => {
    const readings = snapshot.docs.map(doc => doc.data());
    setReadings(readings); // React state updates → UI re-renders
  });
```

AI insight: This is called the **Observer Pattern**. The database pushes changes to you.

Data Visualization with Recharts

Turning Numbers Into Understanding

Raw Firebase data:

```
{ "timestamp": 1710000000, "co2": 1247, "pm25": 18.3, "voc": 340 }
```

Recharts transforms it into interactive time-series charts:

```
<LineChart data={readings}>  
  <XAxis dataKey="timestamp" tickFormatter={formatTime} />  
  <YAxis domain={[300, 2000]} />  
  <Line dataKey="co2" stroke="#38bdf8" dot={false} />  
  <ReferenceLine y={1000} stroke="red" label="Danger threshold" />  
  <Tooltip />  
</LineChart>
```

Progressive Web App (PWA)

Making a Website Feel Like a Native App — Without Writing Native Code

What makes a PWA?

1. Web App Manifest (`manifest.json`)

- App name, icon, theme color
- Tells the browser "this can be installed to the home screen"

2. Service Worker (JavaScript file running in the background)

- Caches static assets for fast loading
- Enables offline support

PWA — What I Actually Learned

Adding One File to My Existing React App Made It a Mobile App

Before: My Firebase-hosted React dashboard was desktop-only in a browser.

After: With `manifest.json` linked in `index.html`, any phone user can:

- Open the site in Safari or Chrome
- Tap "Add to Home Screen"
- Launch it from their home screen like any other app — full screen, no browser UI, my icon

No App Store. No developer account. No Xcode. No additional code.

AI helped me understand: The browser treats a PWA differently from a normal website once it detects a valid manifest. `display: standalone` removes the browser chrome —

State Management — The Hardest Concept

What AI Helped Me Truly Understand

The problem: How does the CO2 chart know to update when a new reading arrives from Firebase?

Wrong mental model (mine at first):

"The chart polls Firebase every 60 seconds"

Correct mental model (what AI taught me):

Firebase `onSnapshot` → updates React `state` → React re-renders any component that uses that state → only the CO2 chart changes, nothing else

Firebase pushes data

↓

`setReadings(newData)` ← React state setter

What AI Got Wrong — And What I Fixed

AI hallucination I caught:

When I asked Claude how to query Firebase for "last 24 hours" of data, it gave me:

```
// AI's suggestion — doesn't work this way in Firestore  
db.collection("readings").where("timestamp", ">", "24hours_ago_string")
```

The actual fix I found:

```
const yesterday = new Date();  
yesterday.setDate(yesterday.getDate() - 1);  
const yesterdayTimestamp = firebase.firestore.Timestamp.fromDate(yesterday);  
  
db.collection("readings")  
  .where("timestamp", ">", yesterdayTimestamp) // Firebase Timestamp, not string  
  .orderBy("timestamp", "asc")
```

Lesson: AI knows the concept but sometimes gets the exact API wrong. Always test

Results — What I Built and Deployed

Dashboard features built and working:

- ✓ Real-time sensor cards — CO2, PM2.5, VOC, Temperature, Humidity
- ✓ Time-series line charts for all 5 sensors (24h / 7d / 30d views)
- ✓ Threshold indicators (red line at CO2 1000 ppm, PM2.5 35 $\mu\text{g}/\text{m}^3$)
- ✓ Purifier status card (ON/OFF + which sensor triggered it)
- ✓ Sleep quality rating interface (1–10 input, stored in Firebase)
- ✓ PWA installable on iOS Safari and Android Chrome
- ✓ Publicly accessible without login

What I Learned — Summary

Before	After
"React is just JavaScript"	Understand component model, props, state, re-rendering
"Just setInterval to refresh"	Know why reactive/observer pattern is better for IoT
"Firebase is a database"	Can use real-time <code>onSnapshot</code> subscriptions
"Charts are just plugins"	Can structure data correctly for time-series visualization
"PWA vs native is unclear"	Can choose the right app delivery model for the use case
"AI knows the exact API"	Know to always verify AI-generated code against actual docs

Key Takeaway

Building a real-time IoT dashboard isn't about formatting a webpage — it's about designing a reactive system where the UI automatically responds to a continuous stream of physical-world data. Every architectural decision, from state management to database query structure, flows from that single idea.

The difference between a website and a real-time dashboard is the same as the difference between a photograph and a live video feed. AI helped me see that difference and design for it.

Repository: <https://github.com/shresthamausam07/smart-home-health-monitor>

Learning docs: <https://github.com/shresthamausam07/mausam-shrestha-learning-with-ai>

Thank You

Mausam Shrestha

CSC 494 — IoT | Spring 2026

shrestham2@mymail.nku.edu

Topic 2 complete — Mobile App Development with React/PWA